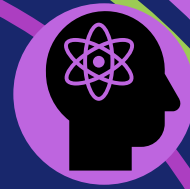
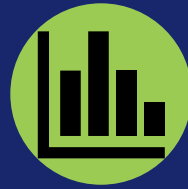
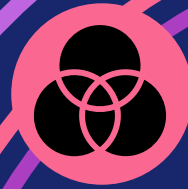


2026年度 前期 データマイニング特論 (I20教室)
第11回 2026年6月25日(木) 13:10~14:40



データマイニング特論

第11回

テキストマイニング

本科目シラバスの確認 3（授業計画）

#	日程	内容
第1回	4/16（木）	ガイダンス、イントロダクション+Python環境構築
第2回	4/23（木）	pandasによるデータ操作の基礎
第3回	4/30（木）	探索的データ分析（EDA）と可視化（オンデマンド）
第4回	5/7（木）	データの前処理と特徴量エンジニアリング
第5回	5/14（木）	分類の基礎 — k近傍法と決定木
第6回	5/21（木）	アンサンブル学習
第7回	5/28（木）	線形分類モデルと回帰
第8回	6/4（木）	モデル評価・選択と不均衡データ
第9回	6/11（木）	クラスタリング手法
第10回	6/18（木）	アソシエーション分析と異常検知
第11回	6/25（木）	テキストマイニング
第12回	7/2（木）	時系列データマイニング
第13回	7/9（木）	深層学習の概観と使いどころ
第14回	7/16（木）	公平性・倫理と再現性
第15回	7/23（木）	まとめ

【Point】 やむなく欠席した場合、資料を確認のうえ不明ところは聞いてください。

1



テキストマイニングとは

非構造化データ(テキスト)を特徴ベクトルに変換し、既習手法で分析する

テキストマイニングとは

【数値・カテゴリデータとの根本的な違い】

非構造化データの分析

これまで：行＝サンプル、列＝特徴量の「表形式（構造化）」データを扱ってきた

テキストは長さも語彙もバラバラで、そのままでは数式・アルゴリズムに乗らない
→ まず「特徴ベクトル」に変換することがすべての出発点

身近なテキスト源	： レビュー・問い合わせ・SNS・論文・物件説明文
ノイズが多い	： 表記ゆれ・誤字・記号・HTML・多言語混在
高次元・疎	： 語彙が数千～数万次元、ほとんどがゼロ

テキストマイニングの全体像

【このパイプラインを一気通貫で扱う】

共通の幹：テキスト → 特徴ベクトル化 → これまで学んだ手法（クラスタリング・分類・類似度）を再利用

前処理

クリーニング
分かち書き

ベクトル化

BoW / TF-IDF

トピック抽出 (LDA)

教師なしで話題を発見

テキスト分類

TF-IDF + 既習の分類器

類似度・検索

コサイン類似度

テキスト前処理と分かち書き

【日本語特有の課題＝語の区切りが無い】

- ①クリーニング : HTML・記号除去
- ②正規化 : 大小・全半角
- ③トークン化 : 語に分割
- ④ストップワード : 汎用語の除去
- ⑤原形化 : 表記の統一

英語：空白で区切れる

“Cozy room near the station”

⇒[Cozy] [room] [near] [the] [station]

単純な `.split()` で済む

日本語：区切りが無い

「駅近くのお部屋」

⇒[駅] [近く] [清潔] [部屋]

形態素解析が必須

(Janome / MeCab / Subachi)

Bag of Words (BoW)

【語の出現回数を並べたベクトルにする】

＜考え方＞

- ・ 語順を捨て、どの語が何回出ただけを数える
- ・ 全文書の語彙が「列」、各文書が「行」
- ・ ほとんどの要素は 0 → 疎 (sparse) 行列
- ・ 語彙数だけ次元が増える → 次元の呪い

＜例：2つの短い文書＞

文書1 = 「駅 近い 静か」

文書2 = 「駅 広い 清潔」

	駅	近い	静か	広い	清潔
文書1	1	1	1	0	0
文書2	1	0	0	1	1

各文書が5次元のベクトルになった

TF-IDF

【「ありふれた語」を割り引き、特徴的な語を重く】

$$\text{tfidf}(t, d) = \text{tf}(t, d) \times \log(N / \text{df}(t))$$

tf：その文書での出現頻度

df：その語が現れた文書数

N：総文書数

<IDF の直感>

- ・ どの文書にも出る語（df 大）は log で小さく → 重みが下がる
- ・ 一部の文書にだけ出る語は重みが上がる
- ・ 結果：その文書「らしい」語が浮かび上がる

<実装上の論点>

- ・ n-gram：連続する語をまとめて1語として扱う
- ・ min_df / max_df で語彙サイズを制御
- ・ 疎行列のまま計算（メモリ効率）

トピックモデリング (LDA) ①

【文書＝トピックの混合、トピック＝単語の分布】

1つの文書：複数トピックの混合比でできている



トピック：観光

各トピックは単語の確率分布： 浅草 / 寺 / 観光 / 下町 / 散歩

トピック：設備

各トピックは単語の確率分布： wifi / キッチン / 洗濯機 / 清潔

トピック：立地

各トピックは単語の確率分布： 駅 / 徒歩 / 便利 / アクセス

トピックモデリング (LDA) ②

【ハードクラスタリングとの違い・トピック数 k 】

クラスタリングとの対比

k-means (ハード)

⇒ 1 文書は 1 クラスタに所属

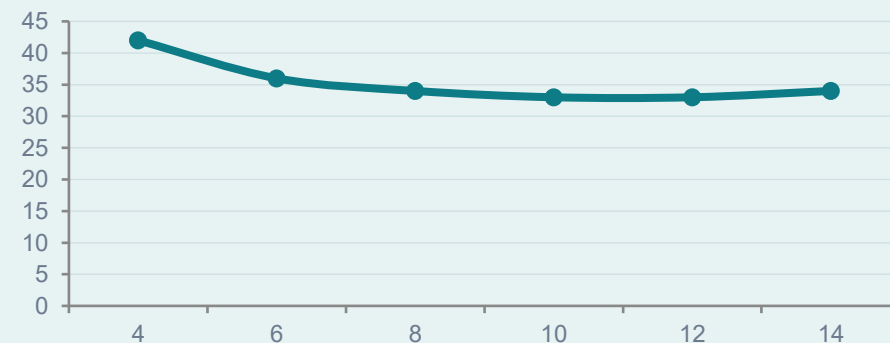
LDA (ソフト)

⇒ 1 文書が複数トピックを比率で持つ

LDA の入力は

TF-IDF ではなく「出現回数」

トピック数 k の選び方



※ perplexity (小さいほど当てはまり良) と解釈しやすさのトレードオフで決める (概念図)

既習手法への接続

【特徴ベクトルにすれば、これまでの手法が全部使える】

コサイン類似度

TF-IDFベクトル間の角度で文書の近さを測る

→ 類似物件検索・レコメンド

テキスト分類

TF-IDF+ナイーブベイズ/ロジスティック回帰

→ 第7-8回の分類・評価へ接続

埋め込み・LLM

単語埋め込み・BERTで文脈を捉える

→ 「深層学習」への布石

まとめ

- ・ テキスト = 非構造化データ。
まず特徴ベクトルに変換するのがすべての出発点
- ・ BoW は出現回数、TF-IDF は「ありふれた語」を割り引いて特徴語を際立たせる
- ・ 日本語は分かち書き（形態素解析）が前処理の要
- ・ LDA は文書をトピックの混合として捉える教師なし手法（入力 は 出現回数）
- ・ 特徴ベクトル化により、分類・クラスタリング・類似度など既習手法へ接続できる

2

本日の演習

実際に解析してみよう！



本日の環境準備

いろいろ試してみよう！

```
# 日本語対応のモジュールをインストールする(その1)
```

```
!pip install japanize-matplotlib
```

```
# 日本語対応のモジュールをインストールする(その2)
```

```
!apt-get install -y fonts-ipafont-gothic
```

```
# janomeをインストールする
```

```
%pip install janome
```

```
# 各ライブラリーをインポートする
```

```
import re
```

```
import numpy as np
```

```
import pandas as pd
```

```
from janome.tokenizer import Tokenizer
```

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
```

```
from sklearn.decomposition import LatentDirichletAllocation
```

```
from sklearn.metrics.pairwise import cosine_similarity
```

```
from sklearn.cluster import KMeans
```

```
from sklearn.naive_bayes import MultinomialNB
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
```

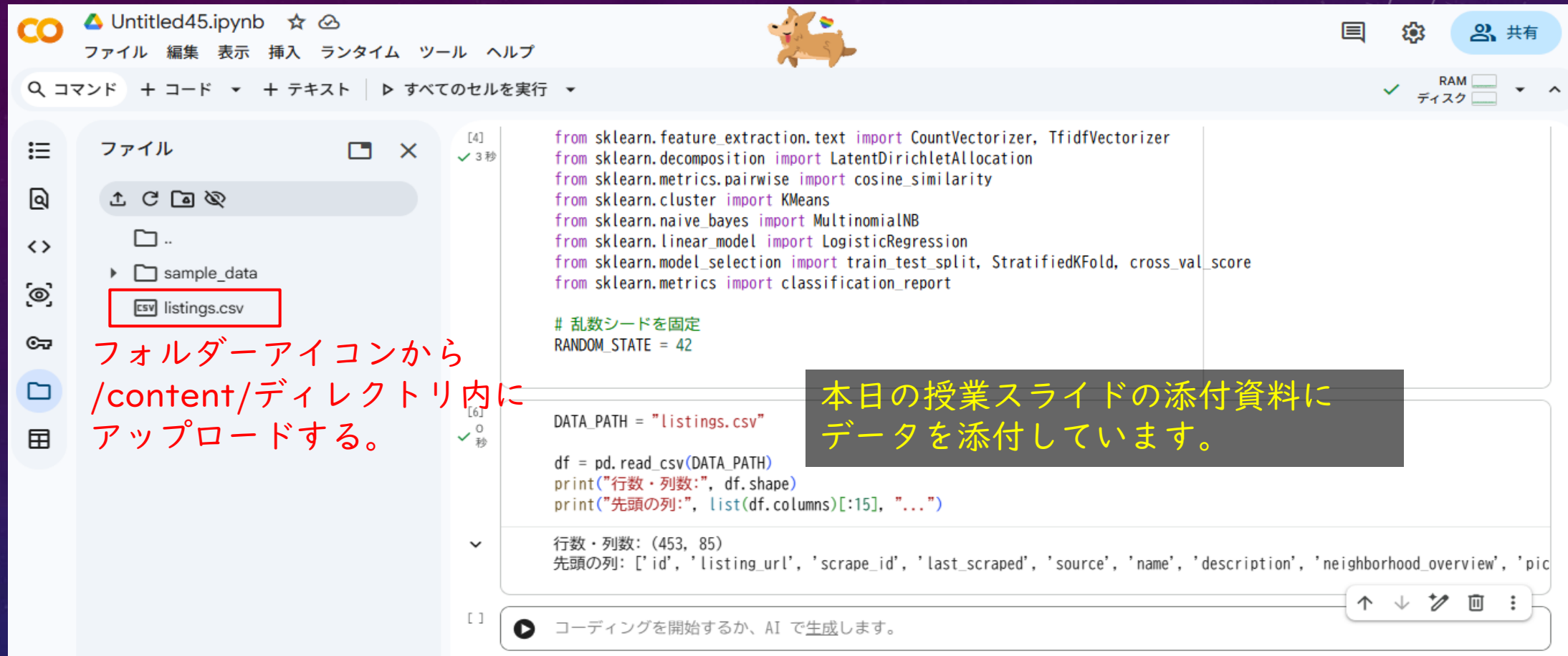
```
from sklearn.metrics import classification_report
```

```
# 乱数シードを固定
```

```
RANDOM_STATE = 42
```


テキストマイニング データの読み込み

いろいろ試してみよう！



The screenshot shows a Jupyter Notebook environment. On the left, a file browser sidebar displays a folder named 'sample_data' containing a file 'listings.csv', which is highlighted with a red box. A red text overlay points to this file, stating: 'フォルダーアイコンから /content/ディレクトリ内にアップロードする。' (Upload from the folder icon to the /content/ directory).

The main notebook area contains two code cells. The first cell imports various machine learning libraries from sklearn. The second cell sets a random seed and defines the data path. Below the code, the output shows the dimensions and first columns of the loaded data.

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.decomposition import LatentDirichletAllocation
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.cluster import KMeans
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.metrics import classification_report

# 乱数シードを固定
RANDOM_STATE = 42

DATA_PATH = "listings.csv"

df = pd.read_csv(DATA_PATH)
print("行数・列数:", df.shape)
print("先頭の列:", list(df.columns)[:15], "...")
```

The output of the second cell is:

```
行数・列数: (453, 85)
先頭の列: ['id', 'listing_url', 'scrape_id', 'last_scraped', 'source', 'name', 'description', 'neighborhood_overview', 'pic']
```

A yellow text box overlay on the right side of the notebook states: '本日の授業スライドの添付資料にデータを添付しています。' (The data is attached to the lecture slide materials).

Inside Airbnb 東京の listings.csv(詳細版を推奨)

```
DATA_PATH = "listings.csv"
```

```
df = pd.read_csv(DATA_PATH)
print("行数・列数:", df.shape)
print("先頭の列:", list(df.columns)[:15], "...")
```

行数・列数: (453, 85)

先頭の列: ['id', 'listing_url', 'scrape_id', 'la

テキスト列の組み立てと HTML 除去

いろいろ試してみよう！

```
# 詳細版には description / neighborhood_overview がある。存在する列だけ結合する。
```

```
TEXT_CANDIDATES = ["name", "description", "neighborhood_overview"]
text_cols = [c for c in TEXT_CANDIDATES if c in df.columns]
print("テキストとして使う列:", text_cols)
```

```
def strip_html(s: str) -> str:
    """Airbnb の説明文には <br /> などのタグや実体参照が混ざるため除去する."""
    s = re.sub(r"<[>]+>", " ", s)      # HTMLタグ
    s = re.sub(r"&[a-zA-Z]+;", " ", s)   # 実体参照 (& など)
    s = re.sub(r"%s+", " ", s)          # 連続空白を1つに
    return s.strip()
```

```
df["text"] = ""
for c in text_cols:
    df["text"] = df["text"] + " " + df[c].fillna("").astype(str)
df["text"] = df["text"].map(strip_html)
```

```
# エリア列(区)。詳細版は neighbourhood_cleansed、要約版は neighbourhood。
```

```
if "neighbourhood_cleansed" in df.columns:
    AREA_COL = "neighbourhood_cleansed"
elif "neighbourhood" in df.columns:
    AREA_COL = "neighbourhood"
else:
    AREA_COL = None
print("エリア列:", AREA_COL)
```

```
テキストとして使う列: ['name', 'description', 'neighborhood_overview']
エリア列: neighbourhood_cleansed
```

日本語の分かち書き（形態素解析）①

いろいろ試してみよう！

ポイント:英語は空白で区切れるが、日本語は形態素解析で語に分割する必要がある。

```
_tokenizer = Tokenizer()
```

```
TARGET_POS = {"名詞", "形容詞"} # 内容語のみ採用(動詞は課題で追加)
```

```
DROP_POS_SUB = {"数", "代名詞", "非自立", "接尾", "副詞可能"}
```

ストップワード(汎用語・Airbnb頻出のノイズ語)。各自で拡張してよい。

```
STOPWORDS = {
```

```
    "こと", "もの", "ため", "よう", "それ", "これ", "ここ", "そこ", "とき", "中", "方", "等", "様", "為", "他", "上", "下",  
    "前", "後", "際", "部屋", "お部屋", "物件", "ゲスト", "滞在", "利用", "ホスト",  
    "room", "house", "apartment", "place", "stay", "guest", "host", "tokyo", "japan", "min", "station",
```

```
}
```

#次に続く。。。


```
def tokenize_ja(text: str):
    """名詞・形容詞を原形で抽出し、1文字語・数字・ストップワードを除く。"""
    out = []
    for tok in _tokenizer.tokenize(text):
        parts = tok.part_of_speech.split(",")
        pos_major = parts[0]
        pos_sub = parts[1] if len(parts) > 1 else ""
        if pos_major not in TARGET_POS:
            continue
        if pos_sub in DROP_POS_SUB:
            continue
        base = tok.base_form if tok.base_form != "*" else tok.surface
        base = base.strip().lower()
        if len(base) <= 1:
            continue
        if re.fullmatch(r"[0-90-9]+", base):    # 純粋な数字は除外
            continue
        if base in STOPWORDS:
            continue
        out.append(base)
    return out
```

#次に続く。。。

日本語の分かち書き（形態素解析）③

いろいろ試してみよう！

英語 vs 日本語の対比デモ

```
print("¥n--- 分かち書きデモ ---")
```

```
print("英語(空白分割) :", "Cozy room near Shibuya station, free WiFi".split())
```

```
print("日本語(分かち書き):", tokenize_ja("渋谷駅まで徒歩5分の清潔で明るいお部屋。無料WiFi完備で観光に便利です。"))
```

--- 分かち書きデモ ---

英語(空白分割): ['Cozy', 'room', 'near', 'Shibuya', 'station,', 'free', 'WiFi']

日本語(分かち書き): ['渋谷', '徒歩', '清潔', '明るい', '無料', 'wifi', '完備', '観光', '便利']

全文書を一度だけトークン化

いろいろ試してみよう！

```
# 形態素解析は重いので「先に1回だけ」分かち書きし、空白区切り文字列に変換しておく。  
# 授業では計算時間短縮のためサンプリング。全件で行う場合は SAMPLE_SIZE = None。  
SAMPLE_SIZE = 3000
```

```
work = df.copy() if SAMPLE_SIZE is None else df.sample(  
    n=min(SAMPLE_SIZE, len(df)), random_state=RANDOM_STATE  
).copy()  
work = work[work["text"].str.len() > 0].copy()  
  
work["tokens"] = work["text"].map(tokenize_ja)  
work["joined"] = work["tokens"].map(lambda t: " ".join(t))  
work = work[work["joined"].str.len() > 0].copy()  
print("¥n対象文書数:", len(work))  
print("平均トークン数:", round(work["tokens"].map(len).mean(), 1))
```

対象文書数: 453
平均トークン数: 62.8


```
# 分かち書き済みなので token_pattern=r"%S+" で空白区切りをそのまま語にする。
```

```
count_vec = CountVectorizer(token_pattern=r"%S+", lowercase=False, min_df=5, max_df=0.5)
```

```
X_count = count_vec.fit_transform(work["joined"])
```

```
tfidf_vec = TfidfVectorizer(token_pattern=r"%S+", lowercase=False, min_df=5, max_df=0.5)
```

```
X_tfidf = tfidf_vec.fit_transform(work["joined"])
```

```
vocab = np.array(count_vec.get_feature_names_out())
```

```
print("¥nBoW 行列:", X_count.shape, " 語彙数:", len(vocab))
```

```
sparsity = 1.0 - X_count.nnz / (X_count.shape[0] * X_count.shape[1])
```

```
print(f"疎性(ゼロ要素の割合): {sparsity:.4f}")
```

```
# 高頻度語 と 高TF-IDF語 の違い
```

```
freq = np.asarray(X_count.sum(axis=0)).ravel()
```

```
top_freq = vocab[np.argsort(freq)[::-1][:20]]
```

```
print("高頻度語 Top20 :", list(top_freq))
```

```
tfidf_sum = np.asarray(X_tfidf.sum(axis=0)).ravel()
```

```
top_tfidf = vocab[np.argsort(tfidf_sum)[::-1][:20]]
```

```
print("高TF-IDF語 Top20:", list(top_tfidf))
```

BoW 行列: (453, 773) 語彙数: 773

疎性(ゼロ要素の割合): 0.9489

高頻度語 Top20 : ['is', 'you', 'home', 'bedroom', 'l

高TF-IDF語 Top20: ['is', 'home', 'you', 'bedroom', 'l

エリア別の特徴語抽出（区ごとの TF-IDF 平均）

いろいろ試してみよう！

```
if AREA_COL is not None:
    print("¥n--- エリア別 特徴語(TF-IDF平均 上位) ---")
    top_areas = work[AREA_COL].value_counts().head(6).index.tolist()
    for area in top_areas:
        mask = (work[AREA_COL] == area).values
        if mask.sum() < 10:
            continue
        mean_tfidf = np.asarray(X_tfidf[mask].mean(axis=0)).ravel()
        top = vocab[np.argsort(mean_tfidf)[::-1][:10]]
        print(f"[{area}] ({mask.sum()}件): " + " / ".join(top))
else:
    print("エリア列が無いためスキップ")
```

--- エリア別 特徴語(TF-IDF平均 上位) ---

[SIXTH WARD] (106件): from / park / center / square / washington / state / heart / historic / street / is
[TENTH WARD] (49件): bedroom / private / is / new / hospitals / st / home / located / queen / med
[SECOND WARD] (43件): state / downtown / historic / street / apt / center / plaza / empire / mvp / you
[THIRTEENTH WARD] (39件): is / home / renovated / our / bedroom / bed / located / floor / mi / centrally
[NINTH WARD] (36件): private / home / on / shared / no / suite / victorian / located / have / bedroom
[THIRD WARD] (33件): downtown / located / is / palace / newly / from / all / bedroom / city / nys

```
# 重要:LDA は「出現回数(カウント)」を入力にする。TF-IDF ではない。
N_TOPICS = 8
lda = LatentDirichletAllocation(
    n_components=N_TOPICS, learning_method="batch",
    max_iter=20, random_state=RANDOM_STATE,
)
doc_topic = lda.fit_transform(X_count)    # 文書×トピック の確率分布

def show_topics(model, feature_names, n_top=10):
    for k, comp in enumerate(model.components_):
        top = feature_names[np.argsort(comp)[::-1][:n_top]]
        print(f"トピック{k:2d}: " + " / ".join(top))

print(f"¥n--- LDA トピック (k={N_TOPICS}) ---")
show_topics(lda, vocab, n_top=10)
work["topic"] = doc_topic.argmax(axis=1)
print("¥nトピック別の文書数:")
print(work["topic"].value_counts().sort_index().to_string())

# トピック数 k の比較(perplexity は小さいほど当てはまり良。ただし解釈性とのトレードオフ)
print("¥n--- トピック数 k の比較(perplexity) ---")
for k in [4, 6, 8, 10, 12]:
    m = LatentDirichletAllocation(
        n_components=k, learning_method="batch", max_iter=20, random_state=RANDOM_STATE
    ).fit(X_count)
    print(f"k={k:2d} perplexity={m.perplexity(X_count):.1f}")
```

LDA によるトピック抽出②

--- LDA トピック (k=8) ---

トピック 0: parking / free / fi / wi / br / fully / comfy / it / kitchen / equipped
 トピック 1: parking / is / friendly / space / off / street / pet / entry / floor / tv
 トピック 2: on / private / is / queen / bedroom / bed / kitchen / large / new / home
 トピック 3: home / kitchen / at / bedroom / located / family / bath / fully / bathroom / you
 トピック 4: from / is / bedroom / minutes / you / home / restaurants / located / street / or
 トピック 5: your / you / cozy / downtown / breakfast / convenience / business / home / perfect / re
 トピック 6: state / historic / empire / plaza / is / downtown / renovated / unit / apt / bed
 トピック 7: center / from / is / medical / square / modern / home / an / st / garden

トピック別の文書数:

topic	
0	18
1	23
2	73
3	52
4	102
5	30
6	86
7	69

--- トピック数 k の比較(perplexity) ---

k= 4 perplexity=431.3
 k= 6 perplexity=423.9
 k= 8 perplexity=423.9
 k=10 perplexity=421.8
 k=12 perplexity=429.2

接続1：コサイン類似度で「似た物件」を検索

いろいろ試してみよう！

```
print("\n--- コサイン類似度による類似文書検索 ---")
query_idx = 0
sims = cosine_similarity(X_tfidf[query_idx], X_tfidf).ravel()
order = np.argsort(sims)[::-1]
order = order[order != query_idx][:5]
name_col = "name" if "name" in work.columns else "text"
print("クエリ:", str(work.iloc[query_idx][name_col])[:60])
for idx in order:
    print(f" 類似度 {sims[idx]:.3f} : " + str(work.iloc[idx][name_col])[:60])
```

--- コサイン類似度による類似文書検索 ---

クエリ: Downtown ALB ・ Hot Tub ・ Game Room ・ Free Parking

類似度 0.285 : 4 bedroom house in Albany, 4 minutes from AMC

類似度 0.281 : ★ BEAUTIFUL 2 BED 2 BATH w/ SOAKER TUB + W/D★

類似度 0.278 : Downtown Albany 1 Bed + Workstation @ Maiden Lane

類似度 0.263 : Downtown Albany 2 Bedroom + Workstation @ The Mark

類似度 0.235 : Central cozy 4 bedroom w/King bed

```
print("\n--- k-means クラスタ × LDA トピック の対応 ---")
km = KMeans(n_clusters=N_TOPICS, n_init=10, random_state=RANDOM_STATE)
work["cluster"] = km.fit_predict(X_tfidf)
print(pd.crosstab(work["cluster"], work["topic"]).to_string())
```

```
--- k-means クラスタ × LDA トピック の対応 ---
topic      0   1   2   3   4   5   6   7
cluster
0          1   2  20  11   9   0   4   2
1          0   0   0   0   0  25   0   0
2          4  12  10  31  49   4   5  23
3          0   6  14   3   6   0   6   4
4          8   0   0   2   2   0  18   1
5          5   3   6   1  33   1  35  39
6          0   0   0   0   0   0  12   0
7          0   0  23   4   3   0   6   0
```

接続3：説明文からの分類（分類・評価の復習）①

いろいろ試してみよう！

ターゲット: レビュースコアがあれば高評価/低評価、無ければ部屋タイプ进行分类。

```
print("¥n--- テキスト分類 ---")
```

```
if "review_scores_rating" in work.columns and ¥
```

```
    pd.to_numeric(work["review_scores_rating"], errors="coerce").notna().sum() > 50:
```

```
    r = pd.to_numeric(work["review_scores_rating"], errors="coerce")
```

```
    work2 = work.assign(_rating=r).dropna(subset=["_rating"]).copy()
```

```
    thr = work2["_rating"].quantile(0.5) # 中央値で2分割(分布を見て調整可)
```

```
    y = (work2["_rating"] >= thr).astype(int).values
```

```
    target_name = f"高評価(>= {thr:.2f}) vs 低評価"
```

```
elif "room_type" in work.columns:
```

```
    work2 = work[work["room_type"].notna()].copy()
```

```
    y = (work2["room_type"] == "Entire home/apt").astype(int).values
```

```
    target_name = "Entire home/apt vs その他"
```

```
else:
```

```
    work2 = None
```

```
    y = None
```

```
    print("分類に使えるターゲット列が無いためスキップ")
```

```
if work2 is not None:
```

```
    Xc = count_vec.transform(work2["joined"]) # 同じ語彙で変換
```

```
    print("タスク:", target_name, " 正例率:", round(float(y.mean()), 3))
```

```
    X_tr, X_te, y_tr, y_te = train_test_split(
```

```
        Xc, y, test_size=0.25, random_state=RANDOM_STATE, stratify=y
```

```
    )
```

```
    clf = MultinomialNB()
```

```
    clf.fit(X_tr, y_tr)
```

```
    print(classification_report(y_te, clf.predict(X_te), digits=3))
```

接続3：説明文からの分類（分類・評価の復習）②

いろいろ試してみよう！

第7-8回の復習:層化K分割で安定評価

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
f1 = cross_val_score(clf, Xc, y, cv=cv, scoring="f1")
print("F1 (5-fold):", f1.round(3), " 平均:", round(float(f1.mean()), 3))
```

どの語がどちらのクラスを示すか(ナイーブベイズの対数確率差)

```
fn = np.array(count_vec.get_feature_names_out())
diff = clf.feature_log_prob_[1] - clf.feature_log_prob_[0]
print("正例を示す語 Top15:", list(fn[np.argsort(diff)[::-1][:15]]))
print("負例を示す語 Top15:", list(fn[np.argsort(diff)[:15]]))
```

--- テキスト分類 ---

タスク: 高評価(>= 4.85) vs 低評価 正例率: 0.522

	precision	recall	f1-score	support
0	0.717	0.702	0.710	47
1	0.736	0.750	0.743	52
accuracy			0.727	99
macro avg	0.727	0.726	0.726	99
weighted avg	0.727	0.727	0.727	99

F1 (5-fold): [0.707 0.575 0.628 0.729 0.643] 平均: 0.656

正例を示す語 Top15: ['closet', 'tub', 'stainless', 'hybrid', 'outdoor', 'abba', 'there', 'jesse',

負例を示す語 Top15: ['pastures', 'me', 'courts', 'regional', 'anytime', 'earl', 'older', 'afford

3

本日の課題

頑張ってください。



【課題】 期限：2026/7/1（水）23：59まで

WebClassの『データマイニング特論』アクセスし、
本日の日付が記載される「課題」を期限内に実施してください。

・ WebClass -> データマイニング特論 -> 【第11回（6/25）】課題

（課題1） 本日の演習で「いろいろ試してみよう！」の中で実施した入出力コードを
html形式でエクスポートし、提出しなさい。

html形式にする手順

- ①ipynb形式でダウンロード（ファイル名の例：aaa.ipynb）
- ②再び「ファイル」へアップロード
- ③以下のコマンドを入力

```
!jupyter nbconvert --to html "aaa.ipynb"
```


【ADSセミナー】

本学の「先端データ科学研究センター」が主催するセミナーが6/25（木）から4週にわたって開催されます。

内容が盛りだくさんなので、**研究室を選択する前の参考になる**かと思います。

事前登録フォームから申し込みあり。

【Point】 普段の授業では聞けない話もあるかも！？

ADSセミナー

対象 東京情報大学
全学部・学年/大学院生/教職員

会場 1号館 201教室

【第1回】 6/25（木） 16:30 - 18:00

講師： 釣井 達也 先生
総合情報学科 准教授（数理情報研究ユニット）



量子ウォークに潜む群構造について

量子ウォークはランダムウォークの量子版と解釈される数学モデルであり、その振る舞いはランダムウォークと大きく異なります。本セミナーでは、量子ウォークの簡単な具体例を通してランダムウォークとの違いを観察します。特に、量子ウォークに潜む群構造に着目し、この群構造を活用して、量子ウォークの特徴的な挙動のひとつである「周期」を求める方法について紹介します。

【第2回】 7/2（木） 16:30 - 18:00

講師： 早稲田 篤志 先生
総合情報学科 准教授（情報セキュリティ研究ユニット）



個人情報保護と機械学習

プライバシーに関する情報は解析して使用することで有益な情報を得られることが期待できます。一方、漏洩等で外部に漏れると大きな問題となることがあります。そこで、基本的なプライバシー保護の指標を紹介し、機械学習との関係を考察します。

【第3回】 7/9（木） 16:30 - 18:00

講師： 西村 明 先生
総合情報学科 教授（情報メディア学系）



音響法科学（オーディオ・フォレンジクス）

音響法科学は、音響学の原理と技術を、犯罪捜査・法廷および不正等の検知のために活かす研究と学問を指します。話者識別、雑音除去と音声強調、録音属性推定に大別され、本講演では、前二者を概観したのち、AI音声の判別を含む後者の研究について概説します。

【第4回】 7/16（木） 16:30 - 18:00

講師： 小早川 睦貴 先生
総合情報学科 准教授（生命情報研究ユニット）



身体情報から読みとる心の働き

心理学的研究で心を読み取る際には、さまざまな人間の「反応」を測ることが不可欠です。一般的には反応時間や正答率、表情など目に見える反応を使いますが、発汗や視線など、目に見えない反応などを使って心を読む心理学的研究を紹介します。

<参加登録>

右のQRコードを読み取り、参加登録フォームへアクセスの上、必要事項を記入してお申し込みください。



受付期間： 5/26（火）～ 7/16（木）

<https://forms.office.com/r/JRmJe3qNmh>

早期研究体験プロジェクト
（対象：学部1・2年次）

本プログラムに興味のある学生は、先端データ科学研究センターまでお問い合わせください。

随時
募集中

<お問い合わせ>

東京情報大学 総合情報学部
先端データ科学研究センター
ads@rsch.tuis.ac.jp

